

Real-Time Scheduling Simulator

Server-Based Algorithms for Mixed Periodic-Aperiodic Workloads

GitHub: <https://github.com/shahabafshar/Scheduler>



Shahab Afshar

CPR E 458/558: Real-Time Systems, Fall 2025

Department of Electrical and Computer Engineering

Instructor: Dr. G. Manimaran

Problem Statement

Real-time systems must satisfy timing constraints in addition to functional correctness.

Real-World Example: Automotive Engine Control Unit (ECU)

- An ECU must handle both predictable and unpredictable workloads:
 - Periodic tasks: Engine control (10ms), ABS monitoring (5ms), fuel injection (20ms)
 - Aperiodic tasks: Driver button presses, diagnostic requests, error handling
 - Challenge: How to guarantee BOTH periodic AND aperiodic tasks meet deadlines?

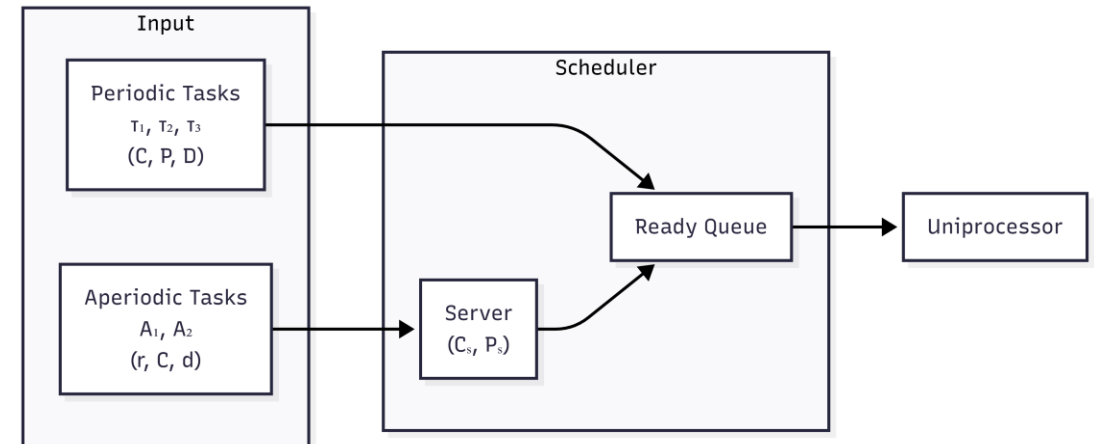
The Gap in Current Tools

- Theoretical analysis provides utilization bounds but doesn't show temporal behavior
- Manual schedule construction is tedious and error-prone
- RTOS testing requires significant development effort before exploring alternatives
- Engineers need to explore algorithm behavior BEFORE committing to implementation

Research Questions

This project investigates three research questions:

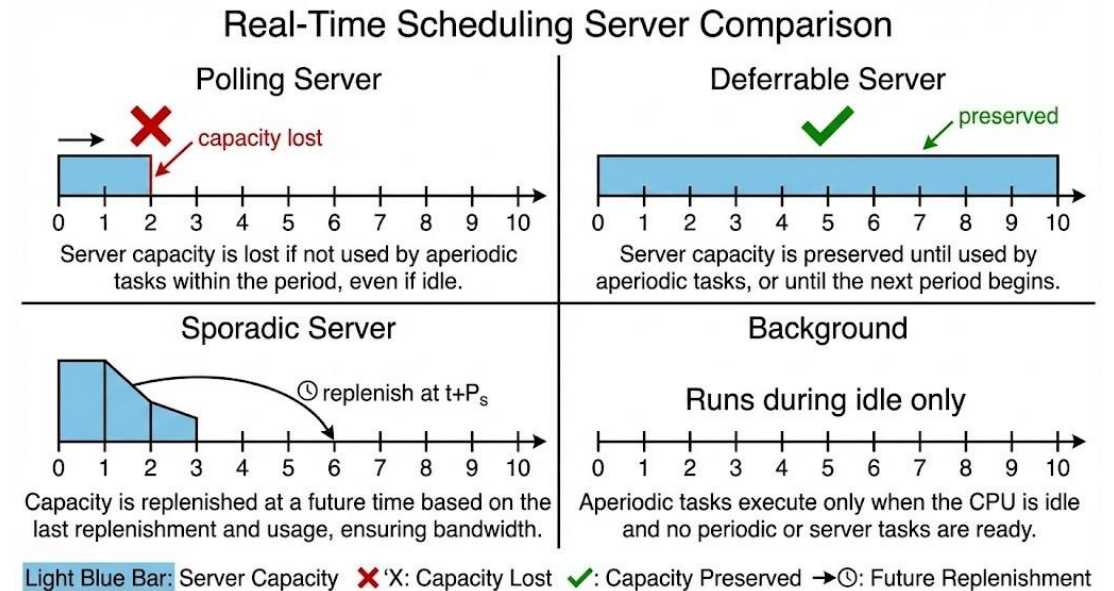
- **RQ1: Faithful Implementation**
 - Can a discrete-event simulator faithfully implement server capacity management policies (Polling, Deferrable, Sporadic)?
- **RQ2: Visualization Effectiveness**
 - Does interactive visualization (Gantt charts, metrics) help users understand algorithm differences?
- **RQ3: Parameter Exploration**
 - Can parameter exploration (varying server capacity C_s and period P_s) reveal optimal configurations?



Solution: Four Server-Based Scheduling Algorithms

Server-based scheduling dedicates CPU capacity to aperiodic tasks while protecting periodic task deadlines:

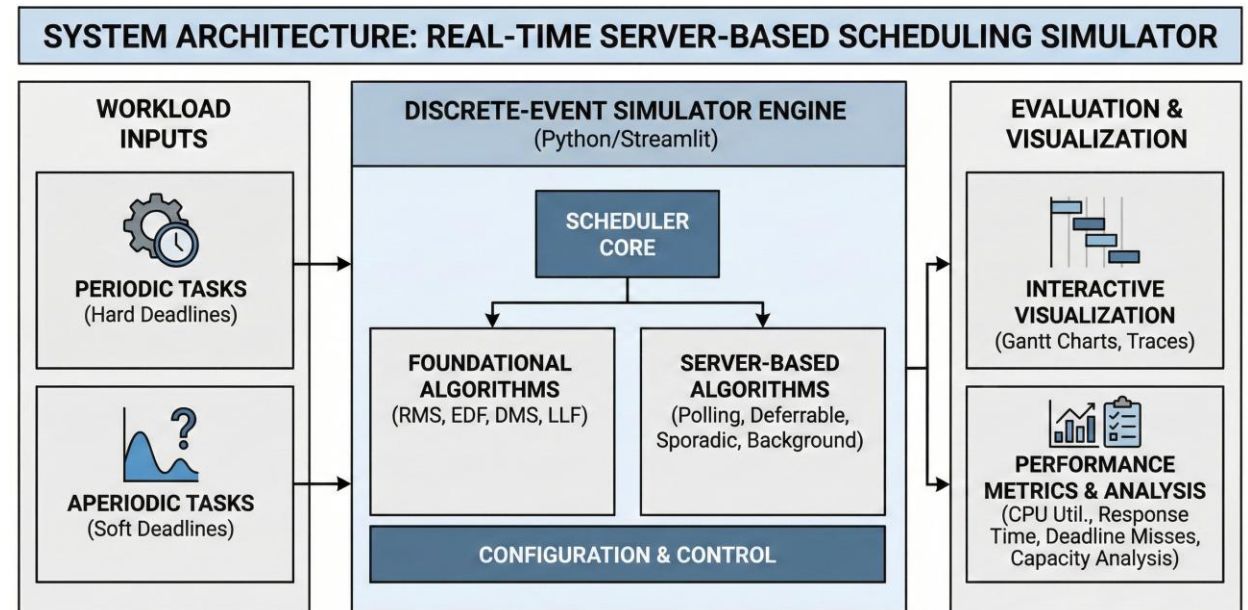
- **Polling Server**
 - Capacity is LOST if no aperiodic tasks are ready when server executes. Simple but wasteful.
- **Deferrable Server**
 - Capacity is PRESERVED until end of server period. Better response but affects periodic schedulability.
- **Sporadic Server**
 - Capacity replenishment scheduled at $t+P_s$ after consumption. Best response while maintaining RMS guarantees.
- **Background Scheduler**
 - Aperiodic tasks run ONLY during CPU idle time. Simplest but worst response time.



Interactive Simulation Platform

A web-based simulator built with Python and Streamlit providing:

- **Core Features**
 - Discrete-event simulation with exact algorithm implementation
 - Interactive Gantt charts showing task execution and server events
 - Real-time parameter exploration via sliders (Cs, Ps, task parameters)
 - 21 preset configurations for quick experimentation
 - Schedulability analysis with RMS, EDF, and DMS utilization bounds
- **Supported Algorithms**
 - Basic: RMS, EDF, DMS, LLF (Liu & Layland, 1973)
 - Server-Based: Polling, Deferrable, Sporadic, Background
 - Advanced: Precedence-constrained, Overload handling, HVDF



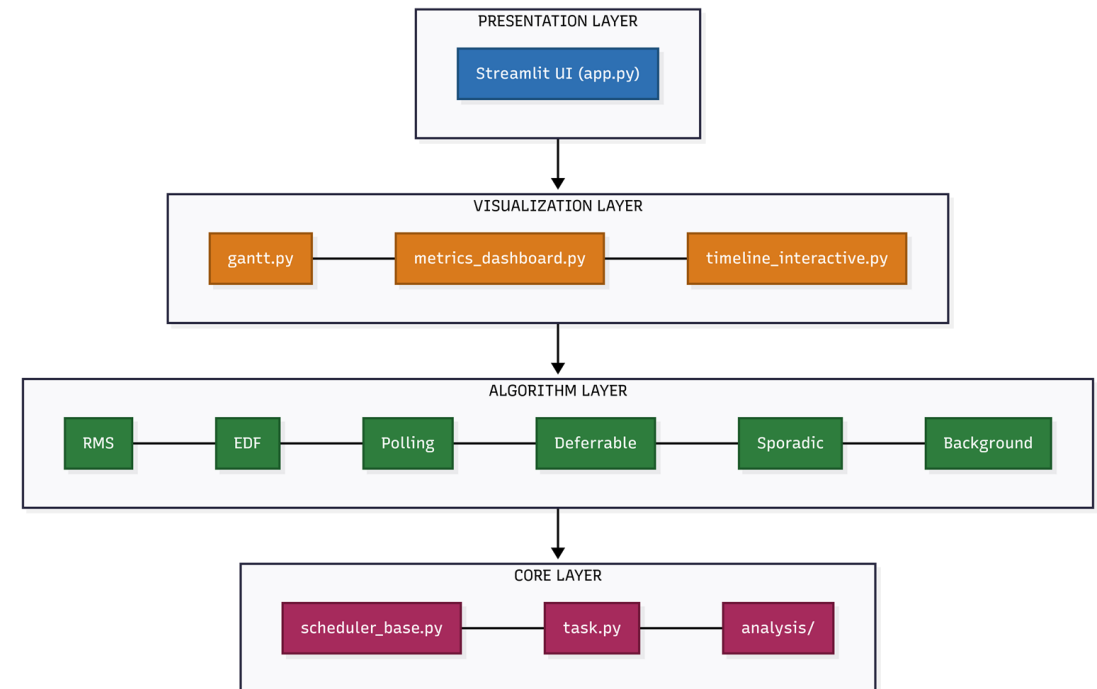
Software Architecture

Layered Design with Clear Separation of Concerns

The architecture separates the simulation engine from visualization and UI, enabling independent testing and extension.

Technology Stack

- **Core Logic (Python 3.10+)**
 - Pure simulation engine with no UI dependencies. Implements all scheduling algorithms and produces ScheduleResult objects.
- **Visualization (Plotly)**
 - Creates interactive Gantt charts, priority timelines, and metrics dashboards from ScheduleResult data.
- **Web UI (Streamlit)**
 - Provides the interactive dashboard with configuration panels, result displays, and export functionality.
- **Data Handling (Pandas)**
 - Manages task tables, CSV/JSON export, and data transformation for visualization.



Design Pattern: Template Method

Key Insight

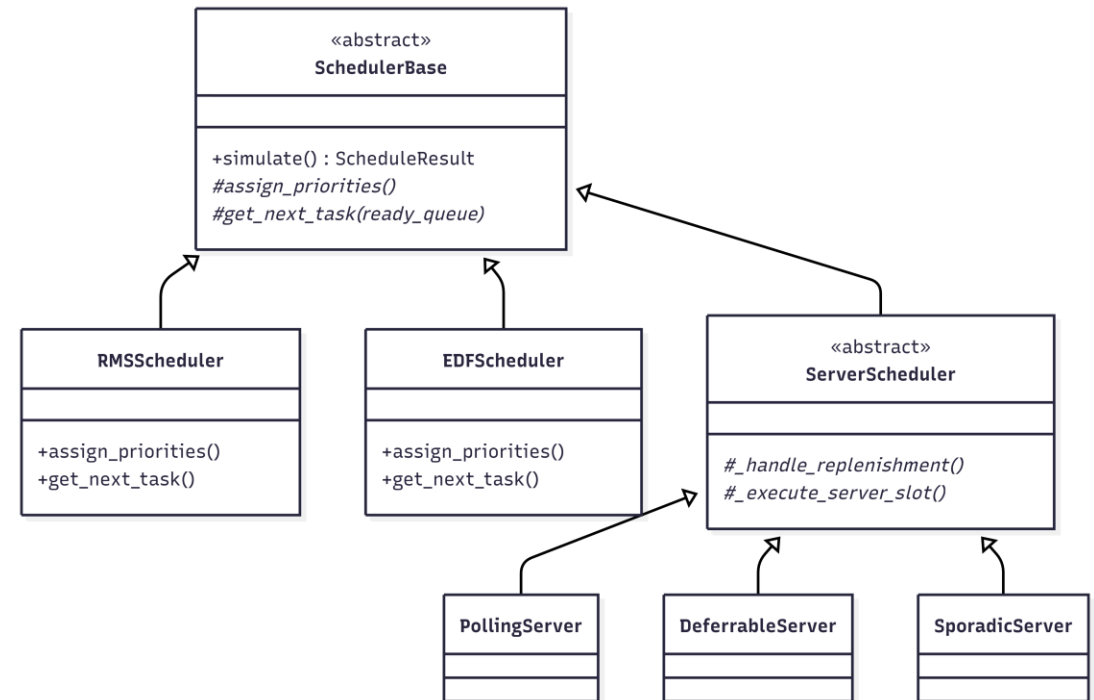
- All schedulers share the same simulation loop (time advancement, ready queue management, preemption handling). Only priority assignment and task selection differ.

Implementation

- SchedulerBase implements the complete simulate() method. Subclasses override only two methods:
 - assign_priorities(): How to assign priorities to tasks
 - get_next_task(ready_queue): Which task to run next

Server Capacity Management

- The key difference between servers is what happens when the server has CPU priority but no aperiodic tasks are ready:
 - Polling: server_remaining = 0 (capacity lost)
 - Deferrable: return False (capacity preserved until period end)
 - Sporadic: Schedule replenishment at current_time + Ps



Results: Correctness Validation (RQ1)

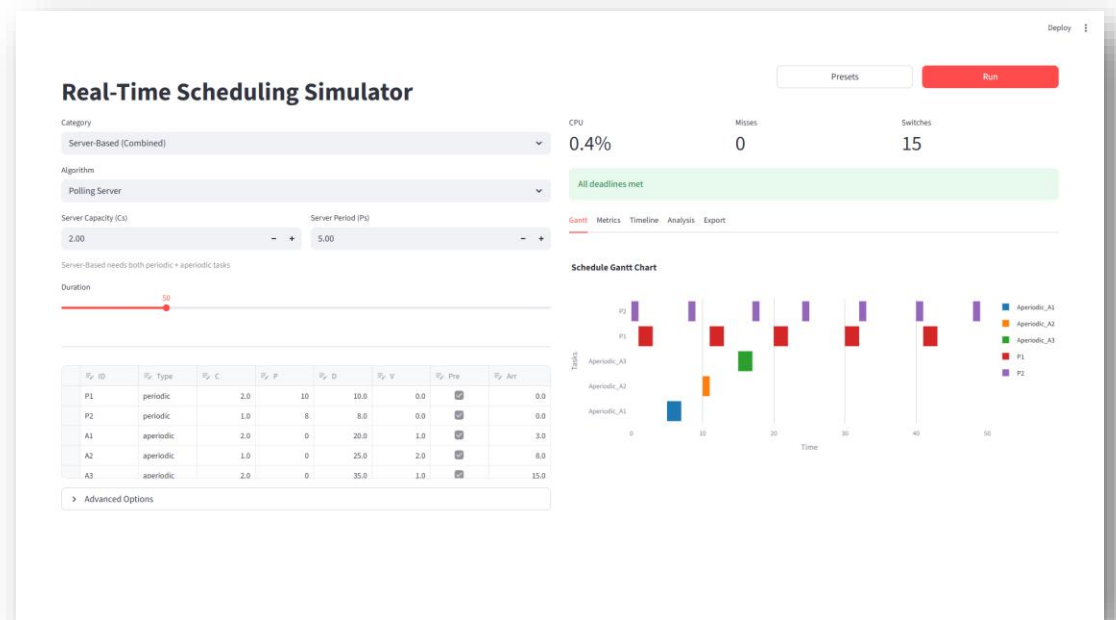
Can the simulator faithfully implement server capacity management?

Verification Method

Each server type produces characteristic events in the timeline. Correctness is verified by observing expected event patterns:

- **Polling Server:** **[Correct]**
 - capacity_lost events appear when server executes with no aperiodic tasks ready
- **Deferrable Server:** **[Correct]**
 - deferred events show capacity preservation; no capacity_lost events
- **Sporadic Server:** **[Correct]**
 - replenish events scheduled exactly at consumption_time + Ps
- **Background Scheduler:** **[Correct]**
 - Aperiodic execution only during idle intervals (no periodic tasks ready)

Answer: YES - The simulator correctly implements all four server algorithms as verified by behavioral event analysis.



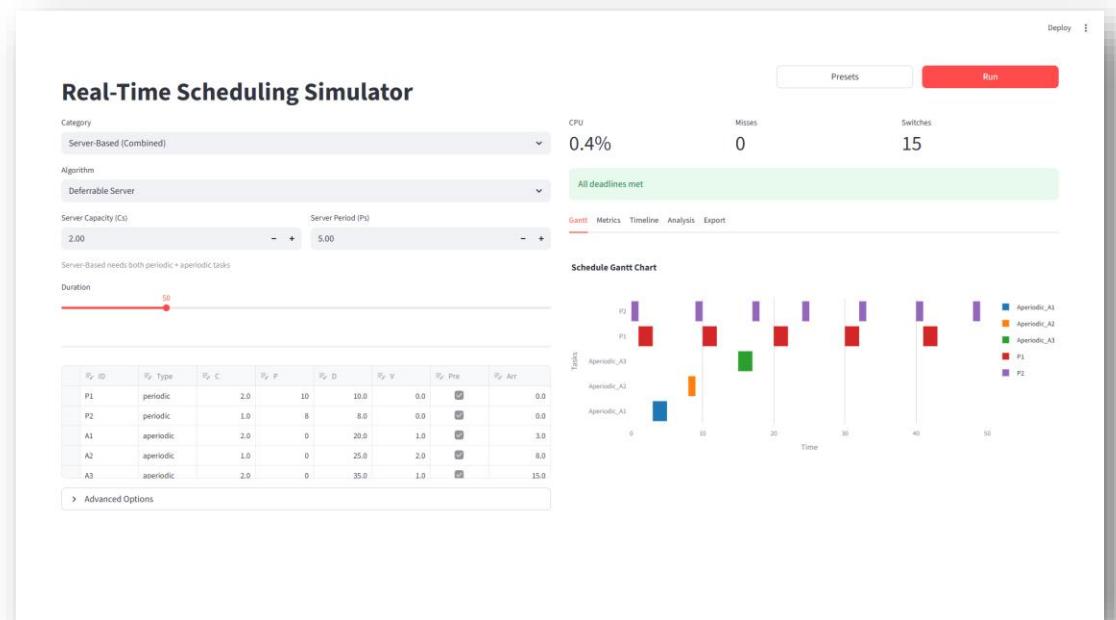
Results: Visualization Effectiveness (RQ2)

Does interactive visualization help users understand algorithm differences?

Visualization Features Implemented

- **Color-coded task bars**
 - Periodic tasks (blue), Aperiodic tasks (orange), Server capacity (green) are visually distinct
- **Deadline markers**
 - Red triangles mark absolute deadlines for each task instance
- **Arrival markers**
 - Green circles indicate task arrival times
- **Server event labels**
 - Events like 'replenish', 'deferred', 'capacity_lost' labeled directly on timeline
- **Hover tooltips**
 - Hovering over any bar shows task ID, remaining computation time, and event details
- **Interactive zoom/pan**
 - Users can zoom into specific time regions for detailed analysis

Answer: YES - Users can visually compare when capacity is lost (Polling) vs. preserved (Deferrable) vs. replenished (Sporadic).



Results: Parameter Sensitivity (RQ3)

Can parameter exploration reveal optimal configurations?

Experiment Setup

- Workload: Two periodic tasks P1(C=1, P=10), P2(C=1, P=15) and one aperiodic task A1(C=8, arrives at t=0). Server period Ps=5 fixed, varying capacity Cs.

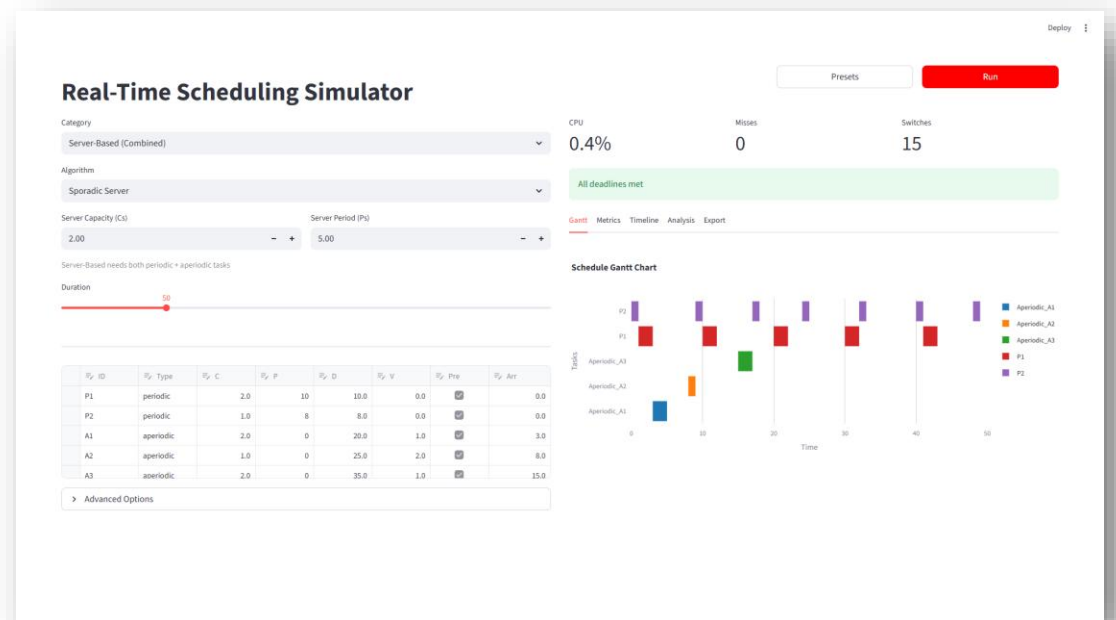
Results: Server Capacity Effect on Response Time

- Cs=2: Response Time = 17 time units (4 replenishment cycles needed)
- Cs=4: Response Time = 9 time units (2 replenishment cycles needed)
- Cs=8: Response Time = 8 time units (1 replenishment cycle needed)

Key Finding

- Increasing server capacity (Cs) from 2 to 8 reduces aperiodic response time by 53% (17→8). This insight cannot be obtained from utilization formulas alone.

Answer: YES - Parameter exploration via sliders enables rapid discovery of optimal server configurations.



Conclusions

Research Question Answers

- **RQ1: Faithful Implementation**
 - The simulator correctly implements capacity management for all four server algorithms, verified by observing expected event patterns (capacity_lost, deferred, replenish).
- **RQ2: Visualization Effectiveness**
 - Interactive Gantt charts clearly show when capacity is lost vs. preserved vs. replenished. Users can visually compare algorithm behavior side-by-side.
- **RQ3: Parameter Exploration**
 - Varying server capacity C_s from 2→8 demonstrated response time reduction from 17→8 time units, enabling discovery of optimal configurations.

Key Contributions

1. Open-source simulator implementing 4 server-based algorithms
2. Interactive Gantt charts displaying capacity management events
3. 21 preset configurations from course literature for verified correctness
4. Parameter exploration via sliders for optimal configuration discovery
5. Schedulability analysis with RMS, EDF, and DMS utilization bounds

Limitations & Future Work

Current Limitations

- **Single-processor only**
 - The simulator currently supports uniprocessor scheduling. Multi-core scenarios with task migration and partitioning are not yet implemented.
- **No resource contention modeling**
 - Priority Inheritance Protocol (PIP) and Priority Ceiling Protocol (PCP) are implemented in the codebase but not yet integrated into the main simulation loop.
- **Limited user validation**
 - RQ2 assessment is based on feature implementation. A formal user study would provide stronger evidence of visualization effectiveness.
- **Simplified execution model**
 - The simulator does not model I/O delays, cache effects, interrupt latency, or context switch overhead beyond basic counting.

Future Directions

- Multi-core extension with partitioned and global scheduling algorithms
- Resource protocol integration (PIP/PCP) for shared resource scenarios
- RTOS trace import to compare with FreeRTOS/Zephyr execution
- Formal verification using model checking for schedulability guarantees

Learning Achieved

Technical Skills Developed

- **Real-Time Systems Concepts**
 - Deep understanding of server-based scheduling (Polling, Deferrable, Sporadic), RMS utilization bounds $n(2^{1/n}-1)$, schedulability analysis, and capacity management trade-offs.
- **Software Engineering Patterns**
 - Applied Template Method pattern for extensible scheduler architecture, Strategy pattern for composable priority policies, and event-driven simulation design.
- **Tools & Technologies**
 - Python for discrete-event simulation, Streamlit for rapid web UI development, Plotly for interactive visualizations, Pandas for data handling.

Key Lessons Learned

1. Simulation reveals temporal behavior that utilization formulas cannot show
2. Visualization is essential for understanding scheduling dynamics
3. Parameter exploration enables informed design decisions before implementation
4. Modular architecture (Template Method) dramatically simplifies adding new algorithms

References

1. [1] B. Sprunt, L. Sha, and J. Lehoczky, "Aperiodic task scheduling for Hard-Real-Time systems," *Real-Time Syst*, vol. 1, no. 1, pp. 27-60, June 1989, doi: 10.1007/BF02341920.
2. [2] J. K. Strosnider, J. P. Lehoczky, and L. Sha, "The deferrable server algorithm for enhanced aperiodic responsiveness in hard real-time environments," *IEEE Transactions on Computers*, vol. 44, no. 1, pp. 73-91, 1995, doi: 10.1109/12.368008.
3. [3] M. Spuri and G. Buttazzo, "Scheduling aperiodic tasks in dynamic priority systems," *Real-Time Systems*, vol. 10, no. 2, pp. 179-210, Mar. 1996, doi: 10.1007/BF00360340.
4. [4] C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment," *J. ACM*, vol. 20, no. 1, pp. 46-61, Jan. 1973, doi: 10.1145/321738.321743.



Thank You

Shahab Afshar

PhD Student, Computer Engineering

Department of Electrical and Computer Engineering

Project Repository: Available upon request